

Paterni ponašanja

Paterni ponašanja (Behavioral Design Patterns)

1.1. Analiza trenutnog stanja i potreba za uvođenjem paterna ponašanja

Trenutna implementacija sistema "InterTrips" (posebno unutar **RezervacijaController**) oslanja se na proceduralni pristup unutar akcija kontrolera. Na primjer, unutar metode

KreirajRezervaciju nalazi se logika za:

- Validaciju podataka i kupona.
- Izračunavanje konačne cijene (bazna cijena pomnožena sa brojem putnika + popust).
- Logiku grananja za plaćanje karticom u odnosu na plaćanje na rate (petlja koja generiše RatePlacanja).

Ovakav dizajn krši **Single Responsibility Principle (SRP)**, jer kontroler obavlja previše različitih zadataka. Također, krši se i princip **Open/Closed Principle (OCP)**, s obzirom na to da bi uvođenje novog načina plaćanja (npr. PayPal) ili novog algoritma za popuste zahtijevalo direktnu izmjenu postojećeg koda unutar kontrolera. Kako bi se postigao labaviji spoj (loose coupling) i omogućila skalabilnost, u sistem je potrebno uvesti arhitektonske paterne ponašanja.

1.2. Prijedlog i opravdanje implementacije specifičnih paterna

A) Strategy Pattern

- **Gdje se primjenjuje:** Unutar podsistema za plaćanje i kalkulaciju cijena.

Trenutno sistem provjerava uslov **model.MetodaPlacanja == "rates"** kako bi odlučio koju logiku izvršavanja treba primijeniti. Uvođenjem *Strategy paterna*, kreira se zajednički interfejs *IPaymentStrategy* sa metodom *ProcessPayment(PaymentDetails details)*. Klase *CardPaymentStrategy* i *InstallmentPaymentStrategy* bi enkapsulirale ovu logiku. Na ovaj način, dodavanje novih metoda plaćanja ne zahtijeva izmjenu postojećeg koda.

B) Observer Pattern

- **Gdje se primjenjuje:** Unutar podsistema za upravljanje kapacitetima i asinhrono slanje notifikacija.

Svaka uspješna rezervacija izaziva lančane reakcije (smanjenje broja soba/sjedišta, promjena statusa paketa u 'Rasprodan', slanje potvrđujućeg e-maila klijentu). Umjesto da kontroler ručno poziva repozitorije, uvodi se Observer patern. *RezervacijaService* preuzima ulogu subjekta (Subject) koji nakon uspješne uplate emituje događaj *ReservationConfirmedEvent*.

Klase poput *HotelCapacityObserver*, *FlightCapacityObserver* i *NotificationService* postaju posmatrači (Observers) koji asinhrono reaguju na emitovani događaj.

C) State Pattern

- **Gdje se primjenjuje:** Upravljanje životnim ciklusom rezervacije preko *StatusRezervacije* (Kreirana, Potvrđena, Otkazana).

Otkazivanje rezervacije zavisi od njenog trenutnog stanja.

Implementacijom State paterna, kreira se apstraktna klasa *ReservationState*, dok stvarna stanja postaju zasebne klase (*CreatedState*, *ConfirmedState*, *CancelledState*). Svaka klasa sama implementira logiku za akcije koje su u tom stanju dozvoljene i upravlja prelazima (tranzicijama) u naredno validno stanje.

D) Template Method Pattern

Gdje se primjenjuje: Podsistem za autentifikaciju, registraciju i validaciju različitih tipova korisničkih naloga (Klijent, Agent/Vodič, Administrator).

Proces registracije i kreiranja profila unutar "InterTrips" aplikacije dijeli identične bazične korake: unos podataka, primarnu validaciju polja, provjeru dupliranja e-mail adrese u bazi, hashing šifre. Uvođenjem ovog paterna, kreira se apstraktna bazna klasa

UserRegistrationTemplate koja fiksira strukturu algoritma. Specifični koraci se prepuštaju podklasama (*ClientRegistration*, *AgentRegistration*), čime se sprječava dupliranje koda i garantuje uniforman sigurnosni protokol tokom registracije.